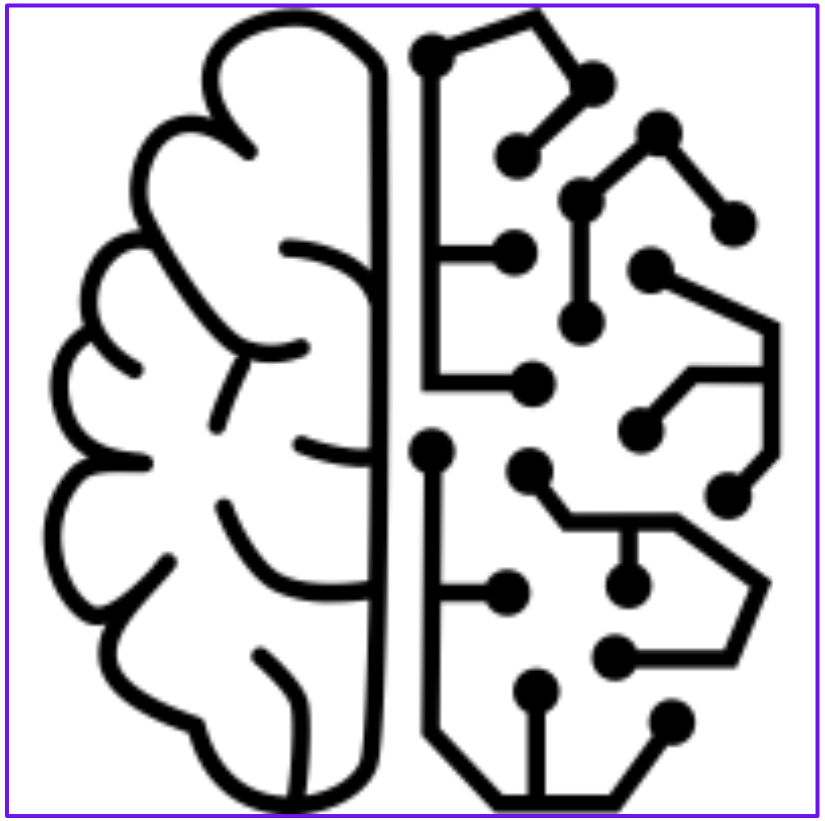

# Application web pour visualiser les données

## M2



NEXA Digital School

## Compétences visées

- RGPD dans le contexte IA (minimisation, finalité, consentement)
- Ethics by Design (transparence, explicabilité, intelligibilité)
- Accessibilité numérique (RGAA/WCAG)
- Préparation des données
- Atténuation des biais

NEXA Digital School

# Ressources

NEXA Digital School

# Programme du Jour

- Matin (9h - 12h30)
  - Introduction aux enjeux (RGPD, Éthique, Accessibilité)
  - Préparation des données financières
- Après-midi (13h30 - 17h00)
  - Modélisation supervisée
  - Atténuation des biais algorithmiques
- Prérequis
  - Python
  - Pandas
  - Intro apprentissage supervisé

# ENJEUX RÉGLEMENTAIRES ET ÉTHIQUES

## Règlement Général sur la Protection des Données

En vigueur depuis mai 2018, s'applique à toute organisation traitant des données de citoyens européens

➢ **Pourquoi c'est crucial en ML ?**

- Les modèles ML traitent souvent des données personnelles
- Les décisions automatisées impactent les droits des individus
- Risques de réidentification même avec données anonymisées
- Conservation des données d'entraînement = stockage de données personnelles

*Note : Amendes jusqu'à 20M€ ou 4% du CA mondial en cas de non-conformité*

## Pourquoi parler de RGPD et d'éthique ?

- **Le contexte réel**
  - 2018 : Amazon abandonne son outil de recrutement par IA (biais anti-femmes)
  - 2019 : Apple Card accusée de discrimination dans l'octroi de crédit
  - 2020 : Algorithme de notation des examens au UK favorise les écoles privées
- **Les risques financiers**
  - Amendes RGPD : jusqu'à 4% du CA mondial ou 20M€
  - Procès pour discrimination
  - Atteinte à la réputation

**Les 3 piliers de notre approche**

1. RGPD (Règlement Général sur la Protection des Données)
   a. Le cadre légal européen

2. Ethics by Design
   a. L'éthique dès la conception

3. Accessibilité numérique
   a. L'inclusion de tous les utilisateurs

# RGPD : Les principes clés pour l'IA

## Les Principes RGPD Essentiels pour le ML

- **Minimisation**
  - Ne collecter que les données strictement nécessaires
    - Bon : Prédire prix maison → surface, localisation, nombre de pièces
    - Mauvais : + origine ethnique, religion, orientation sexuelle
  - **Exemple finance** : Pour un prêt, on a besoin du revenu, pas du détail des dépenses de santé
- Finalité
  - Utiliser les données uniquement pour l'objectif déclaré
  - Définir clairement l'usage : "Prédire l'approbation de prêt"
  - Interdit : Données collectées pour santé → réutilisées pour marketing
- Droit à l'oubli
  - Capacité de supprimer les données d'un utilisateur sur demande
- Consentement
  - Informer clairement l'utilisateur
  - Obtenir un consentement explicite
  - Permettre le retrait du consentement

# RGPD : Droits des personnes

## Les droits essentiels

- **Droit d'accès :** Voir ses données
- Droit à l'oubli : Supprimer ses données
- Droit à l'oubli
- Droit à la portabilité : Récupérer ses données
- Droit d'opposition : Refuser un traitement automatisé

## Impact sur notre application

- Stocker les données de manière extractible
- Logger les prédictions avec justifications
- Prévoir des fonctions de suppression

# Ethics by Design : Les 5 piliers

**1. Transparence**
L'utilisateur doit savoir qu'une IA est utilisée
- Afficher clairement "Ce système utilise l'IA"
- Expliquer le processus de décision

**2. Explicabilité**
Comprendre POURQUOI une décision a été prise
- Utiliser SHAP, LIME, ou feature importance
- Exemple : "Prêt refusé car : revenu insuffisant (60%), historique crédit (30%), ratio d'endettement (10%)"

**3. Intelligibilité**
Rendre le modèle compréhensible pour les non-experts
- Éviter le jargon technique
- Visualisations claires

# Ethics by Design (suite)

**4. Fiabilité**

Le modèle doit être robuste et précis

- Validation croisée rigoureuse
- Monitoring continu en production
- Détection de dérive du modèle

**5. Sécurité**

Protection contre les attaques et fuites

- Chiffrement des données sensibles
- Protection contre l'adversarial ML
- Gestion sécurisée des secrets (API keys)

# Accessibilité numérique : RGAA/WCAG

NEXA Digital School

## Accessibilité numérique : RGAA/WCAG

- **Pourquoi c'est important ?**
  - 15% de la population en situation de handicap
  - Obligation légale depuis 2005 (France)
  - Améliore l'expérience pour TOUS

**Les 4 principes WCAG**

- **1. Perceptible**
  - Textes alternatifs pour images
  - Contraste suffisant (ratio 4.5:1 minimum)
  - Ne pas utiliser uniquement la couleur
- 2. Utilisable
  - Navigation au clavier
  - Temps suffisant pour lire/interagir

## Accessibilité (suite)

- **3. Compréhensible**
  - Langage clair et simple
  - Instructions explicites
  - Messages d'erreur informatifs

- **4. Robuste**
  - Compatible avec les technologies d'assistance
  - Code HTML sémantique
  - Respect des standards

```
# ✅  alt text sur les images
st.image("graph.png", caption="Distribution des revenus")

# ✅ labels clairs sur les inputs
st.number_input("Revenu mensuel net (en €)", min_value=0)

# ✅ messages d'erreur explicites
if income < 0:
    st.error("⚠️ Le revenu ne peut pas être négatif. Veuillez saisir un montant positif.")
```

# TP : Analyse de risques RGPD

**Scénario**
Une banque développe un système d'IA pour approuver automatiquement les prêts immobiliers. Le système collecte :

- Nom, prénom, date de naissance
- Revenus, charges, patrimoine
- Historique bancaire (5 ans)
- Données de géolocalisation des transactions
- Photos du bien immobilier
- Score de crédit
- Données des réseaux sociaux du demandeur

- Questions
  - Identifier 3 problèmes RGPD dans cette collecte
  - Pour chacun, expliquer quel principe est violé
  - Proposer une correction

# Correction TP 1

**Problèmes identifiés**

- Données de géolocalisation des transactions
  - Principe violé : Minimisation
  - Correction : Non nécessaire pour évaluer un prêt immobilier
- Données des réseaux sociaux
  - Principes violés : Finalité + Risque de discrimination
  - Correction : Aucun lien avec la capacité de remboursement
- Historique bancaire de 5 ans
  - Principe violé : Proportionnalité (possiblement excessif)
  - Correction : 2-3 ans suffisent généralement

- Données légitimes
  - Revenus, charges, patrimoine, score crédit, photos du bien

# PRÉPARATION DES DONNÉES

## Notre projet : Loan Approval Prediction

### Contexte métier
Une banque souhaite automatiser partiellement l'évaluation des demandes de prêts personnels.

### Objectif technique
Prédire si un prêt sera approuvé ou refusé en fonction des caractéristiques du demandeur.

### Dataset
**Source :** Kaggle - Loan Approval Prediction
**Taille :** ~600 lignes, 13 colonnes
**Type :** Classification binaire (Approved / Rejected)

### Pourquoi ce dataset ?
- Taille raisonnable pour l'apprentissage
- Problématique réelle et compréhensible
- Permet d'aborder les biais (genre, statut marital)

# Structure du dataset

## Variables disponibles

- Démographiques
  - Gender : Male / Female
  - Married : Yes / No
  - Dependents : Nombre de personnes à charge
  - Education : Graduate / Not Graduate
- Financières
  - ApplicantIncome : Revenu du demandeur
  - CoapplicantIncome : Revenu du co-demandeur
  - LoanAmount : Montant demandé
  - Loan_Amount_Term : Durée du prêt (mois)
- Historique
  - Credit_History : 1 = bon historique, 0 = mauvais
  - Property_Area : Urban / Semiurban / Rural
- Cible
  - Loan_Status : Y (Approved) / N (Rejected)

# Exploration initiale avec Pandas

## Les commandes essentielles

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Charger le dataset
df = pd.read_csv('loan_data.csv')

# Vue d'ensemble
print(df.shape)  # Dimensions
print(df.head())  # Premières lignes
print(df.info())  # Types et valeurs manquantes
print(df.describe())  # Statistiques descriptives

# Distribution de la cible
print(df['Loan_Status'].value_counts())
print(df['Loan_Status'].value_counts(normalize=True))
```

# TP 2 : Exploration du dataset

## Exploration du dataset

- Objectifs
  - Charger le dataset `loan_data.csv`
  - Afficher les 10 premières lignes
  - Identifier les types de variables
  - Compter les valeurs manquantes par colonne
  - Calculer le taux d'approbation global
  - Créer 3 visualisations :
    - Distribution des revenus (histogramme)
    - Taux d'approbation par genre (bar chart)
    - Corrélation entre variables numériques (heatmap)

## Étape 1 : Valeurs manquantes

Stratégies selon la variable

```python
# Variables numériques : médiane ou moyenne
df['LoanAmount'].fillna(df['LoanAmount'].median(), inplace=True)
df['Credit_History'].fillna(df['Credit_History'].mode()[0], inplace=True)

# Variables catégorielles : mode
df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
df['Married'].fillna(df['Married'].mode()[0], inplace=True)

# Ou supprimer si peu de NaN
df.dropna(subset=['Loan_Amount_Term'], inplace=True)
```

- Règle d'or
  - Imputer les NaN seulement sur le train set, puis appliquer la même stratégie au test set.

# Nettoyage (suite)

## Étape 2 : Doublons

```python
# Identifier les doublons
print(f"Nombre de doublons : {df.duplicated().sum()}")

# Supprimer les doublons
df.drop_duplicates(inplace=True)
```

➢ Étape 3 : Outliers (valeurs aberrantes)
Méthode IQR (InterQuartile Range)

```python
def remove_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return df[(df[column] >= lower) & (df[column] <= upper)]

# Appliquer sur les revenus
df = remove_outliers(df, 'ApplicantIncome')
```

# Outliers : Visualisation

## Boxplot pour détecter les outliers

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

# Revenus demandeur
axes[0].boxplot(df['ApplicantIncome'].dropna())
axes[0].set_title('Applicant Income')
axes[0].set_ylabel('€')

# Revenus co-demandeur
axes[1].boxplot(df['CoapplicantIncome'].dropna())
axes[1].set_title('Coapplicant Income')

# Montant du prêt
axes[2].boxplot(df['LoanAmount'].dropna())
axes[2].set_title('Loan Amount')

plt.tight_layout()
plt.show()
```

# Feature Engineering pour la finance

## Créer des variables métier pertinentes

```
# Revenu total du ménage
df['TotalIncome'] = df['ApplicantIncome'] + df['CoapplicantIncome']

# Ratio d'endettement (Loan Amount / Income)
df['LoanAmountToIncome'] = df['LoanAmount'] / df['TotalIncome']

# Mensualité estimée (formule simplifiée)
# EMI = [P x R x (1+R)^N] / [(1+R)^N-1]
# Simplifié : LoanAmount / Term
df['EMI'] = df['LoanAmount'] / df['Loan_Amount_Term']

# Ratio mensualité / revenu
df['EMIToIncome'] = df['EMI'] / df['TotalIncome']

# Logarithme des variables à forte asymétrie
df['Log_LoanAmount'] = np.log(df['LoanAmount'] + 1)
df['Log_TotalIncome'] = np.log(df['TotalIncome'] + 1)
```

# Encodage des variables catégorielles

## Label Encoding vs One-Hot Encoding

**Label Encoding** : Pour variables ordinales

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
df['Education'] = le.fit_transform(df['Education'])
# Graduate = 1, Not Graduate = 0
```

**One-Hot Encoding :** Pour variables nominales

```
df = pd.get_dummies(df, columns=['Property_Area', 'Gender'], drop_first=True)
# Crée Property_Area_Semiurban, Property_Area_Urban, Gender_Male
```

**Règle**

- Ordinale (ordre logique) → Label Encoding
- Nominale (pas d'ordre) → One-Hot Encoding

# TP 3 : Nettoyage et Feature Engineering

## Nettoyage et Feature Engineering

- Objectifs
  - Traiter les valeurs manquantes (stratégie au choix)
  - Détecter et traiter les outliers sur ApplicantIncome
  - Créer les features suivantes :
    - TotalIncome
    - LoanAmountToIncome
    - EMI
    - Log_LoanAmount
  - Encoder les variables catégorielles
  - Sauvegarder le dataset nettoyé

- Livrable
  - Un notebook Jupyter propre avec commentaires
  - Un fichier loan_data_clean.csv

# Correction TP 3 : Points clés

## Valeurs manquantes

```python
# Stratégie recommandée
numerical_cols = ['LoanAmount', 'Loan_Amount_Term', 'Credit_History']
for col in numerical_cols:
    df[col].fillna(df[col].median(), inplace=True)

categorical_cols = ['Gender', 'Married', 'Dependents', 'Self_Employed']
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

## Outliers

```python
# Capping au lieu de suppression
def cap_outliers(df, column, lower_percentile=1, upper_percentile=99):
    lower = df[column].quantile(lower_percentile/100)
    upper = df[column].quantile(upper_percentile/100)
    df[column] = df[column].clip(lower, upper)
    return df

df = cap_outliers(df, 'ApplicantIncome')
```

# MODÉLISATION SUPERVISÉE

## Rappel : Apprentissage supervisé

**Principe**
On dispose de **données étiquetées** (X, y) :

- X = variables explicatives (features)
- y = variable cible (label)

Le modèle apprend la relation X → y sur un **train set**, puis on évalue sa performance sur un **test set** qu'il n'a jamais vu.

**Types de problèmes**

- **Régression** : prédire une valeur continue (ex: prix)
- **Classification** : prédire une catégorie (ex: approuvé/rejeté)

Notre cas : **Classification binaire**

## Train/Test Split

**Pourquoi séparer ?**

- **Train set (70-80%) :** Pour entraîner le modèle
- **Test set (20-30%) :** Pour évaluer la performance réelle

Implémentation

```python
from sklearn.model_selection import train_test_split

# Séparer X (features) et y (cible)
X = df.drop('Loan_Status', axis=1)
y = df['Loan_Status']

# Split 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,  # Pour la reproductibilité
    stratify=y  # Maintient la proportion de classes
)

print(f"Train set: {X_train.shape}")
print(f"Test set: {X_test.shape}")
```

## Validation croisée (Cross-Validation)

**Problème du simple split**
Un seul split peut être chanceux ou malchanceux.

**Solution : K-Fold Cross-Validation**

- Diviser le train set en K parties (folds)
- Entraîner K fois, en utilisant K-1 folds pour train et 1 pour validation
- Moyenner les performances

```python
from sklearn.model_selection import cross_val_score

# 5-fold cross-validation
scores = cross_val_score(model, X_train, y_train, cv=5, scoring='accuracy')

print(f"Accuracy scores: {scores}")
print(f"Mean accuracy: {scores.mean():.3f} (+/- {scores.std():.3f})")
```

## Choix du modèle : Régression Logistique

**Pourquoi commencer simple ?**

- Rapide à entraîner
- Interprétable (coefficients = importance)
- Baseline solide
- Conforme à l'explicabilité (Ethics by Design)

**Implémentation**

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# Normalisation (importante pour la régression logistique)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Modèle
model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(X_train_scaled, y_train)

# Prédictions
y_pred = model.predict(X_test_scaled)
y_pred_proba = model.predict_proba(X_test_scaled)[:, 1]
```

## Alternative : Random Forest

**Avantages**

- Gère les non-linéarités
- Robuste aux outliers
- Feature importance native
- Pas besoin de normalisation

**Inconvénients**

- Moins interprétable (boîte noire)
- Plus lent à entraîner

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,  # Nombre d'arbres
    max_depth=10,      # Profondeur max
    random_state=42,
    class_weight='balanced'  # Compense le déséquilibre de classes
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

## Métriques d'évaluation

### Au-delà de l'accuracy

**Accuracy =** (TP + TN) / Total
- Problème : trompeuse si classes déséquilibrées

**Exemple** : Si 90% des prêts sont approuvés, un modèle qui prédit toujours "Approved" a 90% d'accuracy mais ne sert à rien !

Matrice de confusion
- **TN** (True Negative) : Bien rejeté
- **FP** (False Positive) : Faussement approuvé (risque pour la banque)
- **FN** (False Negative) : Faussement rejeté (frustration client)
- **TP** (True Positive) : Bien approuvé

| | **Prédit Rejected** | **Prédit Approved** |
|---|---|---|
| **Vraiment Rejected** | TN | FP (Erreur Type I) |
| **Vraiment Approved** | FN (Erreur Type II) | TP |

# Métriques (suite)

## Précision et Rappel

**Precision =** TP / (TP + FP)
- "Parmi ceux qu'on a approuvés, combien sont vraiment solvables ?"
- Vise à minimiser les FP

**Recall =** TP / (TP + FN)
- "Parmi ceux qui sont solvables, combien a-t-on approuvés ?"
- Vise à minimiser les FN

**F1-Score =** 2 × (Precision × Recall) / (Precision + Recall)
- Moyenne harmonique (balance entre les deux)

```
from sklearn.metrics import classification_report, confusion_matrix

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

## ROC-AUC : La métrique ultime

**Courbe ROC (Receiver Operating Characteristic)**

- **Axe Y** : True Positive Rate (Recall)
- **Axe X** : False Positive Rate

**AUC (Area Under Curve)**

- 0.5 = modèle aléatoire
- 1.0 = modèle parfait
- 0.7 = bon modèle
- 0.8 = très bon modèle

```python
from sklearn.metrics import roc_auc_score, roc_curve
import matplotlib.pyplot as plt

# Calculer l'AUC
auc = roc_auc_score(y_test, y_pred_proba)
print(f"AUC: {auc:.3f}")

# Tracer la courbe ROC
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}')
plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

## Interprétation du modèle : Feature Importance a vu :

- **Pourquoi c'est crucial ?**
  - Explicabilité (Ethics by Design)
  - Débogage du modèle
  - Insights métier

Régression Logistique : Coefficients

```python
# Coefficients (impact sur le log-odds)
feature_importance = pd.DataFrame({
    'feature': X_train.columns,
    'coefficient': model.coef_[0]
}).sort_values('coefficient', ascending=False)

print(feature_importance)
```

Random Forest : Feature Importance

```python
feature_importance = pd.DataFrame({
    'feature': X_train.columns,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

print(feature_importance.head(10))
```

## SHAP : L'explicabilité avancée

### Qu'est-ce que SHAP ?

- **SHapley Additive exPlanations**
  - Basé sur la théorie des jeux
  - Attribue une contribution à chaque feature pour chaque prédiction
  - Fonctionne avec n'importe quel modèle

Installation et usage

```
!pip install shap

import shap
# Créer l'explainer
explainer = shap.Explainer(model, X_train_scaled)
# Expliquer une prédiction
shap_values = explainer(X_test_scaled)
# Visualiser pour un individu
shap.plots.waterfall(shap_values[0])
# Vue d'ensemble
shap.plots.beeswarm(shap_values)
```

## TP 4 : Entraîner et évaluer le modèle

- Objectifs
  - Charger le dataset nettoyé de ce matin
  - Split train/test (80/20)
  - Entraîner une Régression Logistique
  - Calculer les métriques : accuracy, precision, recall, F1, AUC
  - Afficher la matrice de confusion
  - Tracer la courbe ROC
  - Extraire les 10 features les plus importantes
  - (Bonus) Tester un Random Forest et comparer
- Critères de réussite
  - AUC > 0.70
  - Code documenté
  - Visualisations claires

# BIAIS ALGORITHMIQUES

# Qu'est-ce qu'un biais algorithmique ?

## Définition

Un biais survient quand le modèle discrimine systématiquement un groupe de personnes sur la base de caractéristiques protégées (genre, âge, origine, etc.).

### Sources de biais

- **Données historiques biaisées**
  - Ex: Si historiquement les femmes ont été moins approuvées, le modèle reproduit ce biais
- **Variables proxy**
  - Ex: Le code postal peut être un proxy de l'origine ethnique
- **Conception du modèle**
  - Ex: Optimiser uniquement pour l'accuracy ignore l'équité

# Détecter les biais : Analyse par sous-groupes

## Détecter les biais : Analyse par sous-groupes

Calculer les métriques séparément pour chaque groupe démographique.

```
# Taux d'approbation par genre
for gender in ['Male', 'Female']:
    subset = df[df['Gender'] == gender]
    approval_rate = subset['Loan_Status'].value_counts(normalize=True)['Y']
    print(f"{gender}: {approval_rate:.2%}")

# Prédictions du modèle par genre
for gender in ['Male', 'Female']:
    indices = df[df['Gender'] == gender].index
    subset_pred = y_pred[indices]
    approval_rate = (subset_pred == 'Y').mean()
    print(f"{gender} (modèle): {approval_rate:.2%}")
```

## Indicateur d'équité

Disparate Impact Ratio = (Taux groupe protégé) / (Taux groupe privilégié)

- Ratio < 0.8 → biais significatif (règle "80%")

## Détecter les biais : Visualisation

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Préparer les données
results = pd.DataFrame({
    'Gender': df['Gender'],
    'Actual': y_test,
    'Predicted': y_pred,
    'Income': df['TotalIncome']
})

# Taux d'approbation par genre et tranche de revenu
results['Income_Bracket'] = pd.cut(results['Income'], bins=5, labels=['Very Low', 'Low',
'Medium', 'High', 'Very High'])

pivot = results.groupby(['Gender', 'Income_Bracket'])['Predicted'].apply(lambda x: (x ==
'Y').mean()).unstack()

pivot.plot(kind='bar', figsize=(12, 6))
plt.title('Taux d\'approbation par genre et tranche de revenu')
plt.ylabel('Taux d\'approbation')
plt.xlabel('Genre')
plt.legend(title='Tranche de revenu')
plt.xticks(rotation=0)
plt.show()
```

# Techniques d'atténuation des biais

## 1. Retirer les variables sensibles

```python
# Supprimer Gender, Married (si non nécessaires)
X = df.drop(['Loan_Status', 'Gender', 'Married'], axis=1)
```

Attention aux variables proxy (ex: prénom peut indiquer le genre)

## 2. Rebalancing (Rééquilibrage)

```python
from imblearn.over_sampling import SMOTE

# Suréchantillonner le groupe minoritaire
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
```

## 3. Reweighting (Répondération)

```python
from sklearn.utils.class_weight import compute_sample_weight

# Donner plus de poids aux exemples du groupe sous-représenté
weights = compute_sample_weight(class_weight='balanced', y=y_train)
model.fit(X_train_scaled, y_train, sample_weight=weights)
```

# TP : Détecter et corriger un biais

**Scénario**

*Votre modèle entraîné ce matin prédit les prêts. Vous suspectez un biais de genre.*

- **Objectifs**
  - Calculer le taux d'approbation réel par genre
  - Calculer le taux d'approbation prédit par le modèle, par genre
  - Calculer le Disparate Impact Ratio
  - Si biais détecté (ratio < 0.8), appliquer UNE technique de correction au choix :
    - Retirer la variable Gender
    - Reweighting
  - Ré-entraîner le modèle
  - Vérifier si le biais est corrigé
- Livrable
  - Un court rapport (1 page) avec :
    - Analyse du biais initial
    - Technique appliquée
    - Résultats après correction
    - Trade-off observé (accuracy perdue ?)

# Récapitulatif

**Ce qu'on a couvert**

- RGPD, Ethics by Design, Accessibilité
- Préparation des données (nettoyage, feature engineering)
- Modélisation supervisée (régression logistique)
- Métriques d'évaluation (accuracy, precision, recall, AUC)
- Détection et atténuation des biais

**Demain :**

- Créer une interface web avec Streamlit
- Visualiser les données et le modèle
- Rendre l'application interactive et responsive

**Travail à faire ce soir (optionnel)**

- Finaliser le nettoyage du dataset
- Améliorer le modèle (tester d'autres algos, tuning)
- Lire la doc Streamlit : https://docs.streamlit.io/